Tarea 3

Inteligencia Artificial
Facultad de Ingeniería y Ciencias
Universidad Adolfo Ibáñez
Profesores: Eliana Vivas, Claudio Díaz, Mauricio Figueroa

Fecha Distribución: 02 de octubre de 2026
Fecha Entrega: 25 de octubre de 2026

Objetivo

Aplicar conceptos de **Procesamiento de Lenguaje Natural (NLP)** a un problema de preguntas y respuestas sobre documentos. En particular, se entrenarán representaciones vectoriales de palabras mediante **Word2Vec**, que posteriormente serán utilizadas para implementar búsqueda semántica y un sistema básico de **Retrieval-Augmented Generation (RAG)**.

1. Contexto y problema

Una aplicación del NLP consiste en desarrollar sistemas capaces de buscar información relevante y responder preguntas formuladas en lenguaje natural.

En esta tarea se trabajará con una adaptación de **WikiQA**, dataset compuesto por preguntas y oraciones provenientes de artículos de Wikipedia. Para cada pregunta existe un conjunto de **oraciones candidatas**, algunas de las cuales contienen información que permite responderla.

Ejemplo:

Pregunta: *Who wrote The Old Man and the Sea?*
Candidata 1: *The Old Man and the Sea is a novel published in 1952.* → No responde.
Candidata 2: *The novel was written by Ernest Hemingway.* → Sí responde.
Candidata 3: *The story takes place near Cuba.* → No responde.

Se utilizará el archivo:

WikiQA_Tarea3.tsv

El dataset contiene **29.196 registros correspondientes a 3.043 preguntas**. Cada registro representa una pareja **pregunta–oración candidata**, y la variable Label indica si la oración responde (1) o no (0) a la pregunta. Una misma pregunta aparece, por lo tanto, en múltiples filas. Además, existen preguntas para las cuales ninguna candidata constituye una respuesta correcta.

El corpus se encuentra en **inglés y no deberá ser traducido**.

El objetivo será construir progresivamente un sistema que permita representar preguntas y oraciones mediante embeddings, recuperar las candidatas semánticamente más relacionadas con una pregunta y utilizar posteriormente esta información como contexto para generar una respuesta.

2. Campos del dataset

Campo	Descripción
QuestionID	Identificador de la pregunta.
Question	Pregunta formulada en lenguaje natural.
DocumentID	Identificador del documento de Wikipedia asociado.
DocumentTitle	Título del artículo de Wikipedia.
SentenceID	Identificador de la oración candidata.
Sentence	Texto de la oración candidata.
Label	1 si la oración responde la pregunta y 0 en caso contrario.

Importante: la unidad de observación corresponde a una **pareja pregunta–oración candidata**. Todas las filas asociadas a un mismo QuestionID corresponden a la misma pregunta.

3. Desarrollo de la solución

3.1. Análisis exploratorio

Antes de cualquier transformación, analice el dataset en su estado original:

- Reportar cantidad de registros, preguntas y oraciones candidatas por pregunta.
- Determinar cuántas preguntas poseen al menos una respuesta correcta y cuántas no.
- Analizar la distribución de Label.
- Verificar datos faltantes y duplicados.
- Analizar características básicas del corpus, incluyendo cantidad de palabras y tamaño del vocabulario.

3.2 División de los datos

- **70% entrenamiento**
- **15% validación**
- **15% prueba**

La división deberá realizarse a nivel de **QuestionID**, manteniendo todas las candidatas de una pregunta en el mismo conjunto y procurando conservar aproximadamente la proporción de preguntas con y sin respuesta.

Reporte para cada conjunto el número de registros, preguntas y preguntas con/sin respuesta. Verifique explícitamente que no exista intersección de QuestionID entre los tres conjuntos.

Utilice una semilla aleatoria fija. El conjunto de **prueba deberá permanecer reservado hasta la evaluación final**.

3.3 Preprocesamiento y análisis lingüístico

Defina un pipeline de preprocesamiento que se aplicará de forma idéntica a los conjuntos de entrenamiento, validación y prueba. El conjunto de prueba no deberá utilizarse para tomar decisiones sobre el diseño del preprocesamiento. Cualquier comparación empírica entre alternativas deberá realizarse utilizando únicamente entrenamiento y validación.

El pipeline deberá incluir y justificar:

Paso	Descripción
Normalización	Conversión a minúsculas u otra estrategia justificada
Tokenización	Criterio y método utilizado
Puntuación	Tratamiento aplicado a los signos de puntuación
Normalización morfológica	Lematización o stemming, con justificación
Stopwords	Decisión explícita de eliminar o conservar, considerando su posible impacto sobre el contexto aprendido por Word2Vec

Realice además un análisis breve de las categorías gramaticales (Part-of-Speech, POS) presentes en el corpus**.** Este análisis deberá realizarse antes de aplicar stemming, en caso de que dicha técnica sea utilizada, de modo que el etiquetado gramatical se realice sobre palabras en su forma original y no sobre stems que puedan no corresponder a formas léxicas válidas.

3.4. Entrenamiento y análisis de Word2Vec

Utilizando **exclusivamente el conjunto de entrenamiento**, entrene **5 configuraciones** de Word2Vec, considerando modificaciones de hiperparámetros tales como:

- Arquitectura: CBOW o Skip-gram.
- Dimensión del embedding (vector_size).
- Ventana de contexto (window).
- Frecuencia mínima (min_count).
- Número de épocas (epochs).

Para cada modelo, registre los hiperparámetros utilizados y el tamaño del vocabulario. Además, analice cualitativamente los embeddings obtenidos utilizando **al menos cinco palabras** y sus palabras más similares.

Finalmente, analice la cobertura del vocabulario aprendido. Se considerará como palabra fuera del vocabulario (*Out-of-Vocabulary*, OOV) a todo token presente en validación que no pertenezca al vocabulario Word2Vec aprendido exclusivamente con entrenamiento.

Calcule la proporción de tokens OOV en validación como:

$$\text{OOV Rate} = \frac{\text{número de tokens OOV en validación}}{\text{número total de tokens en validación}}$$

Reporte este valor para cada configuración y comente brevemente su relación con min_count y la cobertura del vocabulario.

3.5. Representación de textos y búsqueda semántica

Word2Vec produce un vector para cada palabra. Para comparar preguntas con oraciones completas, construya una representación vectorial utilizando como base el **promedio de los embeddings de las palabras conocidas** del texto.

Implemente una función que permita obtener esta representación tanto para preguntas como para oraciones y documente cómo se tratan las palabras OOV.

La similitud entre una pregunta q y una oración s deberá calcularse utilizando **similitud coseno**:

$$\text{sim}(q, s) = \frac{q \cdot s}{\|q\|\|s\|}$$

Para cada pregunta, calcule la similitud con sus oraciones candidatas y ordénelas de mayor a menor similitud. El resultado será un **ranking de las candidatas** según su relación semántica con la pregunta.

4. Evaluación del sistema de recuperación

Las configuraciones Word2Vec deberán compararse utilizando exclusivamente el conjunto de **validación**.

Para las preguntas que poseen al menos una respuesta correcta, calcule:

- **Recall@1**
- **Recall@3**
- **Recall@5**

Recall@k corresponde a la proporción de preguntas para las cuales al menos una oración correcta (Label = 1) se encuentra dentro de las primeras k posiciones del ranking.

Resuma los experimentos en una tabla:

Exp.	*Modelo*	*Dimensión*	*Window*	*Min. Count*	*Épocas*	*Recall@1*	*Recall@3*	*Recall@5*
1	CBOW							
2	Skip-gram							
...	...							

A partir de validación, seleccione y justifique una **configuración final**, considerando los resultados de recuperación, comportamiento de los embeddings y cobertura del vocabulario.

Análisis de errores

Para la configuración seleccionada analice:

- **3 preguntas recuperadas correctamente.**
- **3 preguntas recuperadas incorrectamente.**

Para cada caso muestre la pregunta, principales candidatas recuperadas, similitud coseno, Label y posición de la primera respuesta correcta.

Discuta posibles causas de los errores considerando, entre otros aspectos, el contexto aprendido por Word2Vec, palabras OOV y las limitaciones de representar textos mediante el promedio de embeddings.

5. Extensión: sistema de preguntas y respuestas mediante RAG

Retrieval-Augmented Generation (RAG) es una arquitectura que combina recuperación de información con un modelo generativo. En esta tarea se utilizará una versión básica del siguiente flujo:

Pregunta → Retrieval con Word2Vec → Contexto recuperado → Modelo generativo → Respuesta

El objetivo de esta sección es observar cómo el sistema de recuperación desarrollado durante la tarea puede utilizarse para proporcionar contexto a un modelo generativo. **El foco principal continúa estando en el retrieval construido con Word2Vec; no se requiere entrenar ni modificar el modelo generativo.**

Implemente un sistema básico de **RAG** compuesto por:

1. **Retrieval:** utilizar el sistema Word2Vec desarrollado anteriormente para recuperar un número fijo de las oraciones con mayor similitud, las cuales serán utilizadas para construir el contexto entregado al modelo generativo. **Indique y justifique la cantidad de oraciones seleccionada.**
2. **Generation:** utilizar las oraciones recuperadas como contexto para un modelo generativo. Se puede utilizar, por ejemplo, un modelo de la familia **FLAN-T5** (como google/flan-t5-small) u otro modelo generativo de tamaño comparable que pueda ejecutarse localmente o mediante una API de acceso gratuito.

Para cada pregunta evaluada se deberá poder observar:

- Pregunta.
- Oraciones recuperadas y sus similitudes.
- Contexto construido.

- Respuesta generada.

La etapa de retrieval deberá ser **la desarrollada en esta tarea**. No se podrán utilizar embeddings externos ni frameworks que reemplacen automáticamente el sistema de recuperación.

Seleccione ejemplos que incluyan preguntas **con y sin respuesta correcta** entre sus candidatas y compare:

- Respuesta del modelo generativo **sin contexto recuperado**.
- Respuesta del modelo generativo **utilizando RAG**.

Analice si el contexto mejora la respuesta, qué ocurre cuando se recupera información incorrecta o no existe una respuesta correcta, y si se generan afirmaciones que no están respaldadas por el contexto.

6. Evaluación final y conclusiones

Una vez seleccionada la configuración mediante validación, realice la evaluación final utilizando el conjunto de **prueba**, el cual no deberá haber sido utilizado para tomar decisiones anteriores.

Reporte:

- Recall@1.
- Recall@3.
- Recall@5.

Compare los resultados de validación y prueba y discuta la capacidad de generalización del sistema.

Finalmente, presente las principales conclusiones obtenidas, considerando al menos:

- Diferencias observadas entre las configuraciones Word2Vec.
- Calidad y limitaciones de los embeddings.
- Relación entre embeddings y desempeño del retrieval.
- Diferencias observadas entre Recall@1, Recall@3 y Recall@5 y qué indican respecto de la posición de las respuestas correctas en el ranking.
- Impacto de palabras OOV.
- Limitaciones de representar textos mediante el promedio de embeddings.
- Beneficios y limitaciones observadas al incorporar RAG.

7. Entregable

Se deberá entregar **un único Jupyter Notebook en Python**, ejecutable completamente de inicio a fin.

El notebook deberá contener todo el desarrollo solicitado en las secciones anteriores, incluyendo código, tablas, resultados y análisis mediante celdas Markdown. **No será suficiente presentar únicamente código y resultados numéricos.**

Al final se deberá incluir:

- Resumen de la estrategia utilizada.
- Principales resultados y conclusiones.
- Contribución específica de cada integrante.

8. Consideraciones

- Los grupos podrán estar conformados por un máximo de **3 integrantes**.
- Se deberá fijar y documentar la semilla aleatoria utilizada.
- Se recomienda utilizar **Python 3.11 o 3.12**.
- Se podrán utilizar pandas, numpy, scikit-learn, gensim y bibliotecas auxiliares.
- **No se podrán utilizar embeddings preentrenados o modelos externos de embeddings para reemplazar Word2Vec.**
- El modelo generativo se utilizará únicamente en la etapa RAG y no podrá reemplazar el retrieval desarrollado.
- La tarea deberá poder ejecutarse completamente durante la corrección.
- Por cada día de atraso se descontarán **0,5 puntos** de la nota final.
- La evaluación considerará tanto los resultados como la correcta metodología, interpretación, análisis de errores y conclusiones.

Fuente de los datos

Disclaimer: Los datos utilizados corresponden a una adaptación del dataset **WikiQA**, desarrollado por Microsoft Research para el estudio de sistemas de *Question Answering* y selección de oraciones. El archivo entregado fue preparado a partir de los datos originales de WikiQA. https://www.microsoft.com/en-us/research/publication/wikiqa-a-challenge-dataset-for-open-domain-question-answering/

Referencia:

Yang, Y., Yih, W. T., & Meek, C. (2015, September). Wikiqa: A challenge dataset for open-domain question answering. In Proceedings of the 2015 conference on empirical methods in natural language processing (pp. 2013-2018). https://aclanthology.org/D15-1237/?